Task 5: Model Behavior Testing (Stress Testing of LLM)

Objective

To evaluate the behavior of a Large Language Model under adversarial and challenging prompts by testing its safety, factuality, and instruction-following capabilities.

Libraries Used

- google-genai – for Gemini API interaction
- python-dotenv – to manage environment variables
- json – for handling structured output
- os – for file handling
- time – to measure response latency

Tools

- VS Code
- Python
- Gemini API

Steps Performed

1. Loaded API key securely using dotenv
2. Created a loop to accept dynamic user prompts
3. Sent prompts to Gemini model for response generation
4. Measured latency for each response
5. Implemented dynamic evaluation logic for:
    - Safety detection
    - Factuality verification
    - Instruction-following validation
6. Generated structured JSON output for each test

1. Displayed real-time results to the user

2. Stored all results in results.json file

3. Generated failure_report.md for identified failures

Errors Faced During Execution

• API quota error (429) – request limit exceeded

• Incorrect evaluation – all outputs marked as Pass initially

• Hallucination detection issue – weak logic failed to detect fake information

• Instruction-following issue – model blindly followed malicious prompts

• Generic logic limitation – hardcoded conditions failed for custom inputs

Solutions Implemented

- Added exception handling for API quota errors
  • Improved evaluation logic to detect:
- Harmful responses
- Hallucinated outputs
- Prompt injection attacks
  • Replaced hardcoded checks with dynamic evaluation logic
  • Validated constraints like sentence count and specific output format
  • Ensured proper JSON structure for output consistency
  • Added fallback failure states to maintain minimum reporting

Conclusion

The system successfully evaluates Large Language Model behavior under stress conditions. It identifies key weaknesses such as hallucinations, safety risks, and instruction-following failures.

This task provides practical understanding of:

• Model reliability

• Failure detection

• Real-world AI testing techniques

It enhances the ability to build robust and trustworthy AI systems.